Visual Computing for Industry, Biomedicine, and Art

μGPT: A Minimal Transformer Language Model

--Manuscript Draft--

Manuscript Number:	
Full Title:	μGPT: A Minimal Transformer Language Model
Article Type:	Original Article
Funding Information:	
Abstract:	Large Language Models (LLMs) have transformed modern Artificial Intelligence due to their remarkable capacity to comprehend and produce natural language. Yet, the majority of existing systems rely on extensive frameworks, significant GPU usage, and high-level libraries that obscure the mathematical and algorithmic foundations of the Transformer model architecture. This poses a considerable challenge for anyone seeking to grasp the inner workings of GPTs without a strong foundational un derstanding of the subject. Large Language Models (LLMs) have revolutionized contemporary Artificial Intelligence by showcasing impressive abilities in understanding and generating natural language. However, most current implementations depend heavily on large-scale frameworks, GPU-intensive training processes, and highly abstracted libraries that conceal the underlying mathe matical and algorithmic principles of Transformer architectures. This creates a substantial obstacle for students, researchers, and independent developers who are trying to understand the internal workings of Generative Pretrained Transformers (GPTs) from basic principles. This paper presents μGPT, a minimal Trans former based language model developed entirely from scratch using pure Python and NumPy without relying on deep learning frameworks such as PyTorch or TensorFlow. The project re constructs the fundamental elements of GPT-style architectures, which include token embeddings, positional embeddings, scaled dot-product self-attention, residual connections, RMS normal ization, multilayer perceptrons, autoregressive next-token predic tion, custom automatic differentiation, gradient backpropagation, Adam optimization, gradient clipping, temperature sampling, top-k sampling, and nucleus sampling. The system is developed through several progressively enhanced versions, starting from a dependency-free autograd-based prototype to a refined NumPy based Transformer training pipeline. This proposed architecture illustrates how contemporary language modeling principles can be replicated using compact and interpretable implementations while preserving conceptual fidelity to large-scale Transformer systems. Experimental evaluation is conducted on a dataset comprising over 32,000 names and additional textual corpora, where the model effectively learns character-level and token-level sequence generation patterns. The study also examines training stability, optimization strategies, inference quality, computational efficiency, and architectural trade-offs in low-resource CPU only environments. Unlike production-oriented LLM frameworks that prioritize scalability over interpretability, μGPT emphasizes transparency, educational accessibility, and mathematical clarity. The project functions as a minimal GPT implementation focused on research, as well as a teaching framework designed to help understand the intricate workings of Transformer language models in detail. Impact Statement: The main effect of this work is to democra tize the comprehension of Transformer architectures by offering a completely transparent, lightweight, and framework-agnostic GPT implementation. μGPT empowers students, educators, and researchers to explore the entire lifecycle of language model de velopment—from tokenization and self-attention to optimization and autoregressive generation—without the need for specialized hardware or large-scale industrial infrastructure. This initiative promotes explainable and accessible AI education while fostering reproducible research in streamlined language modeling systems.
Corresponding Author:	Sanchayan Ghosh, B.Tech University of Engineering and Management, Jaipur North 24 Pgs, West Bengal INDIA
Corresponding Author E-Mail:	sanchayan.ghosh2022@uem.edu.in
Corresponding Author Secondary Information:	

Corresponding Author's Institution:	University of Engineering and Management, Jaipur
Corresponding Author's Secondary Institution:	
First Author:	Sanchayan Ghosh, B.Tech
First Author Secondary Information:	
Order of Authors:	Sanchayan Ghosh, B.Tech
Order of Authors Secondary Information:	
Suggested Reviewers:	
Opposed Reviewers:	
Additional Information:	
Question	**Response**

μGPT: A Minimal Transformer Language Model

Sanchayan Ghosh
Department of Computer Science and Engineering
University of Engineering and Management, Jaipur
Email: sanchayan.ghosh2022@uem.edu.in

***Abstract*—Large Language Models (LLMs) have transformed modern Artificial Intelligence due to their remarkable capacity to comprehend and produce natural language. Yet, the majority of existing systems rely on extensive frameworks, significant GPU usage, and high-level libraries that obscure the mathematical and algorithmic foundations of the Transformer model architecture. This poses a considerable challenge for anyone seeking to grasp the inner workings of GPTs without a strong foundational understanding of the subject. Large Language Models (LLMs) have revolutionized contemporary Artificial Intelligence by showcasing impressive abilities in understanding and generating natural language. However, most current implementations depend heavily on large-scale frameworks, GPU-intensive training processes, and highly abstracted libraries that conceal the underlying mathematical and algorithmic principles of Transformer architectures. This creates a substantial obstacle for students, researchers, and independent developers who are trying to understand the internal workings of Generative Pretrained Transformers (GPTs) from basic principles. This paper presents μGPT, a minimal Transformer based language model developed entirely from scratch using pure Python and NumPy without relying on deep learning frameworks such as PyTorch or TensorFlow. The project reconstructs the fundamental elements of GPT-style architectures, which include token embeddings, positional embeddings, scaled dot-product self-attention, residual connections, RMS normalization, multilayer perceptrons, autoregressive next-token prediction, custom automatic differentiation, gradient backpropagation, Adam optimization, gradient clipping, temperature sampling, top-k sampling, and nucleus sampling. The system is developed through several progressively enhanced versions, starting from a dependency-free autograd-based prototype to a refined NumPy-based Transformer training pipeline. This proposed architecture illustrates how contemporary language modeling principles can be replicated using compact and interpretable implementations while preserving conceptual fidelity to large-scale Transformer systems. Experimental evaluation is conducted on a dataset comprising over 32,000 names and additional textual corpora, where the model effectively learns character-level and token-level sequence generation patterns. The study also examines training stability, optimization strategies, inference quality, computational efficiency, and architectural trade-offs in low-resource CPU-only environments. Unlike production-oriented LLM frameworks that prioritize scalability over interpretability, μGPT emphasizes transparency, educational accessibility, and mathematical clarity. The project functions as a minimal GPT implementation focused on research, as well as a teaching framework designed to help understand the intricate workings of Transformer language models in detail.**

Impact Statement: The main effect of this work is to democratize the comprehension of Transformer architectures by offering a completely transparent, lightweight, and framework-agnostic GPT implementation. μGPT empowers students, educators, and researchers to explore the entire lifecycle of language model development—from tokenization and self-attention to optimization and autoregressive generation—without the need for specialized hardware or large-scale industrial infrastructure. This initiative promotes explainable and accessible AI education while fostering reproducible research in streamlined language modeling systems.

***Index Terms*—Large Language Models, Transformer Architecture, GPT, Self-Attention, Pure Python, NumPy, Educational AI, Autograd, Minimal Language Model, Neural Networks, Natural Language Processing, Deep Learning, Token Embedding, Language Modeling**

I. Introduction

Large Language Models (LLMs) have greatly progressed the domain of Natural Language Processing (NLP) by allowing machines to execute complex language comprehension, text creation, reasoning, summarization, and conversational activities with impressive efficiency. The introduction of the Transformer architecture by Vaswani *et al.* [1] laid the groundwork for contemporary autoregressive language models by utilizing self-attention mechanisms rather than recurrent computations. Expanding on this framework, the Generative Pretrained Transformer (GPT) series showcased the efficacy of extensive unsupervised pretraining for language modeling and subsequent NLP applications [2]–[4]. These advancements revolutionized current artificial intelligence research and positioned transformer-based language modeling as a leading approach in today's NLP systems.

In spite of the impressive abilities of contemporary LLMs, the majority of large-scale transformer systems necessitate billions of parameters, specialized GPU hardware, significant computational resources, and extensive deep learning frameworks. These demands render such systems challenging to analyze, replicate, and comprehend for educational and experimental objectives. As a result, grasping the underlying mathematical principles and architectural structures of transformer models continues to be a complex task for students, researchers, and independent developers.

Various educational and research-focused initiatives have sought to clarify deep learning principles by utilizing streamlined implementations. Projects such as *micrograd* [5], *nanoGPT* [6], and *makemore* [7] demonstrated that neural network architectures can be built from the ground up using clear and concise codebases. These initiatives emphasize conceptual clarity, interpretability, and a deep understanding of low-level implementations rather than prioritizing industrial-scale performance. Drawing inspiration from these educational

models, this project aims to develop a lightweight transformer-based language model that retains the core principles of GPT architectures while ensuring computational accessibility.

In this paper, we present μGPT (microGPT), A minimal transformer language model created entirely using pure Python and NumPy, without depending on external deep learning frameworks like PyTorch or TensorFlow. This framework aims to deliver an interpretable and educational implementation of autoregressive language modeling, all while ensuring architectural simplicity and efficient CPU execution. In contrast to large-scale industrial systems, this model emphasizes transparency, modularity, and ease of experimentation.

The framework includes several key transformer components, such as token embeddings, positional embeddings, self-attention mechanisms, residual connections, layer normalization, RMS normalization, dropout regularization, and autoregressive decoding. Optimization is performed using the Adam optimizer [8], while normalization techniques are inspired by Layer Normalization [9] and RMSNorm [10]. The architecture also integrates training stabilization strategies such as gradient clipping, learning-rate decay, and dropout regularization [11]. Furthermore, various probabilistic decoding techniques, such as temperature sampling, top-k sampling, and nucleus sampling, are utilized to enhance the quality and diversity of text generation.

A significant aspect of this research is the development of a bespoke automatic differentiation engine that facilitates gradient propagation through dynamically created computational graphs. This feature allows for end-to-end transformer training without relying on external machine learning libraries and offers insights into the mathematical foundations of neural network optimization and backpropagation.

The proposed project progresses through several implementation phases. The early iterations concentrate on dependency-free character-level transformer training using pure Python scalar computations, while subsequent versions incorporate NumPy acceleration, enhanced training stability, conversational inference, checkpoint-based model persistence, and token-level language modeling functionalities. The final framework accommodates both educational experimentation and lightweight research applications on standard CPU hardware.

The main aim of this research is not to rival industrial-scale LLMs, but to create a transparent and interpretable platform for comprehending transformer-based generative language modeling from foundational principles. By reducing architectural complexity while maintaining the core mechanisms of GPT systems, the proposed framework acts as an accessible educational resource for exploring contemporary language model architectures.

The major contributions of this work are summarized as follows:

- Development of a lightweight GPT-style transformer framework implemented entirely in pure Python and NumPy.
- Design of a custom automatic differentiation engine for transformer training and backpropagation analysis.
- Implementation of self-attention, residual learning, RMS normalization, dropout regularization, and autoregressive text generation from first principles.
- Integration of advanced probabilistic decoding methods, including temperature sampling, top-k sampling, and nucleus sampling.
- Support for CPU-efficient training, checkpoint-based model persistence, and conversational inference without external deep learning frameworks.
- Creation of an interpretable educational research platform for studying transformer architectures and generative AI systems.

The rest of this paper is structured in the following manner. Section II addresses the theoretical preliminaries and the foundational aspects of transformers that are pertinent to the proposed framework. Section III describes the architecture and implementation details of μGPT. Section IV explains the training methodology and optimization process. Section V presents experimental observations and generated outputs. Section VI discusses limitations and future improvements, followed by the conclusion in Section VII.

* Our implementation is available at this repository: https://github.com/sanchayan7432/microGPT-A-Minimal-Transformer-GPT-Built-Completely-from-Scratch-with-minimal-computational-overhead.git.

II. Preliminaries

This section presents the theoretical foundations and architectural concepts required to understand the proposed μGPT framework. The implementation is primarily based on transformer-based autoregressive language modeling, lightweight automatic differentiation, and probabilistic text generation mechanisms. The proposed system adopts several principles from modern transformer architectures while maintaining a simplified and educational implementation structure.

A. Transformer Architecture

The Transformer architecture introduced by Vaswani *et al.* [1] replaced recurrent neural computation with self-attention mechanisms, enabling efficient parallel processing and improved long-range dependency modeling. Transformer-based language models process input sequences by learning contextual relationships between tokens through attention operations.

In autoregressive transformer models, the likelihood of producing a sequence of tokens is expressed as:

$$P(x_1, x_2, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_{<t})$$

where x_t represents the current token and $x_{<t}$ denotes all previously generated tokens. This formulation enables sequential next-token prediction during training and inference.

The proposed μGPT framework follows a decoder-only transformer architecture similar to GPT-style models [**?**], [**?**], [**?**], where each token is generated auto-regressively based on prior context.

B. Token Embeddings and Positional Encoding

Language models necessitate the transformation of textual data into numerical formats prior to processing. Within the suggested framework, every token is associated with a dense embedding vector through a trainable embedding matrix.

Given a token index t, the embedding representation is defined as:

$$e_t = W_{embed}[t]$$

where W_{embed} is the token embedding matrix.

As transformers do not naturally maintain the order of sequences, positional embeddings are incorporated into token embeddings to convey positional information:

$$x_t = e_t + p_t$$

where p_t represents the positional embedding associated with token position t.

The combined embeddings are then forwarded into the transformer layers for contextual processing.

C. Self-Attention Mechanism

Self-attention serves as the fundamental element of transformer architectures. It enables the model to assess the contextual significance of each token in relation to other tokens within the sequence.

For an input representation X, query, key, and value vectors are computed as:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

where W_Q, W_K, and W_V are trainable projection matrices.

Attention scores are calculated using scaled dot-product attention:

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where d_k denotes the dimensionality of the key vectors.

The self-attention mechanism allows the model to identify semantic connections among tokens while ensuring computational efficiency.

D. Residual Connections and Normalization

Deep transformer networks utilize residual learning to enhance optimization stability and facilitate gradient propagation. Residual connections merge the initial input with transformed representations:

$$y = F(x) + x$$

where $F(x)$ represents the output of the attention or feed-forward sublayer.

The proposed framework incorporates both Layer Normalization [**?**] and RMS Normalization [**?**] to stabilize activations during training.

Layer normalization is computed as:

$$LN(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

where μ and σ^2 denote the mean and variance of the feature vector.

RMS normalization streamlines the normalization process by adjusting activations based on their root mean square magnitude:

$$RMSNorm(x) = \frac{x}{\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2 + \epsilon}}$$

These normalization methods enhance numerical stability and speed up convergence in the optimization process.

E. Feed-Forward Neural Network

Subsequent to the attention block, transformer architectures utilize a position-wise feed-forward neural network (FFN) to enhance their representational capacity.

The operation of the FFN is defined as:

$$FFN(x) = W_2(\sigma(W_1 x))$$

where W_1 and W_2 are trainable matrices and $\sigma(\cdot)$ represents a non-linear activation function.

The suggested framework mainly employs the ReLU activation function because of its straightforwardness and computational effectiveness.

F. Automatic Differentiation

Training deep neural networks necessitates the effective computation of gradients via backpropagation.

The suggested framework incorporates a tailored automatic differentiation engine, drawing inspiration from educational auto diff systems like micrograd [5].

Each scalar value in the computational graph stores:

- the forward-computed numerical value,
- references to parent nodes,
- local derivatives required for gradient propagation.

Gradients are computed using recursive application of the chain rule:

$$\frac{dL}{dx} = \frac{dL}{dy} \cdot \frac{dy}{dx}$$

where L denotes the loss function.

This system allows for comprehensive transformer training without the need for external machine learning libraries.

G. Optimization Using Adam

The proposed framework uses the Adam optimizer [8] for parameter updates. Adam integrates momentum with adaptive learning-rate estimation to enhance the stability of convergence.

The first-order and second-order moment estimates are computed as:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

where g_t denotes the gradient at step t.

The parameter update rule is given by:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

where η represents the learning rate.

The framework also includes gradient clipping and learning rate decay to enhance training stability.

H. Regularization and Dropout

To reduce overfitting and improve generalization, the framework incorporates dropout regularization [11]. During training, neurons are randomly deactivated with probability p:

$$x' = \begin{cases} 0, & \text{with probability } p \\ \frac{x}{1-p}, & \text{otherwise} \end{cases}$$

Dropout encourages the model to learn more robust feature representations.

I. Probabilistic Text Generation

The proposed framework supports multiple decoding strategies for autoregressive text generation.

Temperature sampling modifies output probabilities according to:

$$P_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

where T denotes the temperature parameter.

Lower temperature values produce more deterministic outputs, while higher values increase randomness.

The framework also implements:

- Top-k sampling,
- Nucleus (top-p) sampling,
- Repetition penalties for improved output diversity.

These decoding strategies improve the quality and variability of generated text during inference.

J. Educational Minimal AI Design Philosophy

In contrast to large-scale industrial LLMs that necessitate extensive infrastructure, the suggested µGPT framework prioritizes transparency, modularity, and accessibility for educational purposes. The design deliberately steers clear of significant dependencies and GPU-centric optimizations to ensure a comprehensive grasp of transformer internals.

The implementation illustrates that core concepts of GPT-style language modeling can be replicated through lightweight CPU-based computations, all while maintaining the crucial mechanisms of contemporary autoregressive transformers.

III. Custom Autograd Engine

A. Custom Autograd Engine

A key element of the suggested µGPT framework is the creation of a bespoke automatic differentiation engine built entirely from foundational principles. Contemporary deep learning frameworks like PyTorch and TensorFlow depend on automatic differentiation systems to calculate gradients throughout the training of neural networks. Nevertheless, these systems frequently obscure the fundamental mathematical operations, complicating the understanding of the internal workings of backpropagation and the construction of computational graphs. To enhance interpretability and educational clarity, the proposed framework features a streamlined scalar-based autograd engine that does not utilize external machine learning libraries.

The custom autograd engine is implemented through a `Value` class that represents scalar numerical values within a dynamically constructed computational graph. Each `Value` object stores:

- the forward-computed scalar value,
- the corresponding gradient value,
- references to parent nodes in the computational graph,
- local derivatives required for gradient propagation.

Mathematical operations including addition, multiplication, exponentiation, logarithms, division, and activation functions generate new computational nodes while maintaining the dependency relationships among variables.

Throughout the forward pass, these operations actively construct a directed acyclic graph (DAG) that illustrates the entire sequence of mathematical computations.

Consider a simple operation:

$$z = x \cdot y$$

The local derivatives are:

$$\frac{\partial z}{\partial x} = y$$

$$\frac{\partial z}{\partial y} = x$$

During backpropagation, gradients are propagated recursively through the graph using the chain rule:

$$\frac{dL}{dx} = \frac{dL}{dz} \cdot \frac{dz}{dx}$$

where L represents the final loss function.

The backward propagation process initiates at the output loss node and moves through the computational graph in reverse topological order. Gradients are gathered for each node based on the local derivatives linked to the operation that generated the node. This system facilitates the automatic calculation of gradients for all trainable parameters within the transformer architecture.

The suggested autograd implementation accommodates various fundamental neural network operations, including:

- scalar addition and subtraction,
- multiplication and division,
- exponentiation,
- logarithmic operations,
- exponential functions,
- ReLU activation,
- gradient accumulation through recursive graph traversal.

In contrast to large scale tensor based automatic differentiation systems, the proposed engine focuses mainly on scalar values to enhance conceptual clarity and interpretability. While this method may be less efficient in terms of computation compared to optimized tensor frameworks, it offers a transparent view of how gradients flow through transformer architectures during the optimization process.

The custom autograd engine is extensively utilized in the initial versions of the uGPT framework, particularly in `microgpt_base.py`, `microgpt1.py`, and `microgpt2.py`. These implementations demonstrate how transformer training, attention computation, and neural optimization can be performed without relying on external deep learning ecosystems.

One of the key benefits of the suggested method is its educational significance. By revealing the inner workings of gradient calculation and the construction of computational graphs, the framework allows both researchers and students to witness firsthand the mathematical principles underlying neural network learning. This level of transparency greatly enhances the understanding of backpropagation, optimization, and the dynamics of transformer training.

The custom autograd engine is inspired by lightweight educational frameworks such as *micrograd* [5], while extending the underlying concepts toward transformer-based autoregressive language modeling.

IV. Design of μGPT

The proposed μGPT framework is designed to create a lightweight, fully interpretable transformer-based language model utilizing solely pure Python and minimal external dependencies. Unlike large scale industrial language models that depend on highly optimized GPU frameworks and extensive distributed infrastructures, μGPT is structured as a compact, research-focused architecture that reveals the internal mathematical operations of transformer learning in a clear and educational way.

The framework develops gradually through various implementations, starting with a completely scalar based transformer engine featuring a custom autograd system, and subsequently advancing to enhanced NumPy based self attention architectures. The overarching design philosophy highlights:

- architectural simplicity,
- mathematical transparency,
- modular transformer construction,
- educational interpretability,
- lightweight CPU-based execution.

The proposed framework consists of several interconnected components, including:

- dataset preprocessing and tokenization,
- embedding generation,
- positional encoding,
- multi-head self-attention,
- feed-forward neural layers,
- normalization layers,
- autoregressive token prediction,
- optimization and gradient computation,
- probabilistic text generation.

A. Overall System Architecture

The entire μGPT pipeline starts with the preprocessing of datasets, where text data is converted into distinct token representations. These tokens are then transformed into trainable vector embeddings and processed through transformer blocks that include self-attention and feed-forward layers. Throughout the training phase, the model learns to predict the next token by utilizing autoregressive language modeling objectives.

The complete workflow of the framework is depicted as follows:

Input Text → Tokenization → Embedding Layer → Self-Attention → Fee

The architecture follows the decoder-only transformer paradigm introduced in GPT-based language models [**?**], [1].

B. Tokenizer and Vocabulary Construction

The tokenizer module transforms raw text sequences into distinct tokens. The framework incorporates two methods of tokenization:

- character-level tokenization,
- word-level regular-expression tokenization.

In the early versions (`microgpt_base.py`, `microgpt1.py`, and `microgpt2.py`), character-level tokenization is used for simplicity and reduced vocabulary size. Each unique character is mapped to a numerical index:

$$\text{stoi}(c) \rightarrow i$$

where c represents a character token and i represents its integer token ID.

The later NumPy-based implementation introduces regex-driven word tokenization:

$$\texttt{tokenize(text)} = \texttt{re.findall(...)}$$

This allows the framework to handle sentence-level datasets and question-answer corpora with greater efficiency.

C. Embedding Layer

Each token is converted into a compact vector representation using trainable embedding matrices. The embedding procedure is defined as:

$$x_t = W_E[t]$$

where:

- W_E denotes the token embedding matrix,
- t represents the token index,
- x_t denotes the embedded token vector.

To preserve sequential information, positional embeddings are added:

$$h_t = x_t + p_t$$

where p_t represents the positional embedding corresponding to token position t.

The unified embedding representation is then normalized through either RMSNorm or LayerNorm, based on the specific implementation version.

D. Multi-Head Self-Attention

The self-attention mechanism serves as the fundamental computational element of the transformer architecture. This framework utilizes scaled dot-product attention, akin to the initial transformer design [1].

For an input sequence X, query, key, and value vectors are computed as:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

The attention scores are calculated using scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where d_k denotes the dimensionality of the key vectors.

The architecture further divides embeddings into multiple attention heads:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, \ldots, head_h)W_O$$

This system allows the model to simultaneously learn contextual dependencies across various representation subspaces.

E. Residual Connections and Normalization

Residual skip connections are incorporated after attention and feed-forward operations:

$$x = F(x) + x$$

where $F(x)$ denotes the output of either the attention layer or the feed-forward layer.

Normalization is applied to stabilize training and improve gradient flow. Earlier implementations utilize RMSNorm [10], while the NumPy-based implementation adopts LayerNorm [**?**].

The RMS normalization operation is defined as:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2 + \epsilon}}$$

F. Feed Forward Network

After the attention mechanism, the model utilizes a position-wise feed-forward neural network that comprises two linear transformations along with a ReLU activation function:

$$\text{FFN}(x) = W_2(\text{ReLU}(W_1 x))$$

This stage improves representational learning capacity and enables non-linear feature extraction.

G. Autoregressive Language Modeling

The training objective follows autoregressive next-token prediction. Given a sequence of tokens:

$$(x_1, x_2, x_3, \ldots, x_n)$$

the model predicts:

$$P(x_t \mid x_1, x_2, \ldots, x_{t-1})$$

The probability distribution over the vocabulary is obtained using softmax:

$$P(x_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

where z_i represents the output logits generated by the model.

Cross-entropy loss is used during optimization:

$$\mathcal{L} = -\log P(x_{\text{target}})$$

H. Optimization Strategy

The framework implements the Adam optimizer [8] for parameter updates. Gradient clipping is additionally employed to prevent exploding gradients:

$$g \leftarrow g \cdot \frac{\tau}{||g||}$$

where τ represents the clipping threshold.

Learning rate decay is incorporated in later implementations to improve convergence stability during long training iterations.

I. Sampling and Text Generation

The inference stage generates autoregressive outputs token-by-token. Multiple probabilistic decoding strategies are implemented across different framework versions:

- temperature sampling,
- top-k sampling,
- nucleus (p) sampling,
- repetition penalty mechanisms.

Temperature scaling modifies output probabilities as:

$$P_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

where T denotes the sampling temperature.

Lower temperature values produce more deterministic outputs, while higher temperatures increase diversity and randomness in generated sequences.

J. Evolution of the Framework

The uGPT project evolves across multiple progressively optimized implementations:

- **microgpt_base.py**: Fully scalar transformer with custom autograd engine.
- **microgpt1.py**: Stabilized transformer with dropout, gradient clipping, and improved optimization.
- **microgpt2.py**: Enhanced training stability with better initialization and nucleus sampling.
- **train.py**: NumPy-accelerated self-attention implementation for faster CPU execution.
- **chat.py**: Interactive inference engine supporting conversational generation using trained checkpoints.

This progressive architectural evolution demonstrates how transformer systems can be incrementally developed from mathematical first principles toward practical language generation systems.

K. Design Objectives

The primary objectives of the proposed μGPT framework are summarized below:

- to provide a minimal yet functional transformer implementation,
- to expose the mathematical foundations of GPT architectures,
- to eliminate dependency on heavy deep learning frameworks,
- to support CPU-only execution environments,
- to improve educational interpretability of transformer learning,
- to demonstrate end-to-end autoregressive language modeling using pure Python and NumPy.

The resulting framework functions as an educational research platform as well as a lightweight experimental transformer architecture, making it ideal for examining the internal workings of generative language models.

V. Implementation

The proposed μGPT framework's implementation emphasizes the development of a lightweight autoregressive language model based on transformers, utilizing pure Python and NumPy, while reducing dependence on external deep learning frameworks. The entire system is structured as a modular architecture that includes training, inference, tokenization, optimization, checkpointing, and conversational generation modules.

The framework undergoes several implementation phases, starting with a fully scalar-based transformer engine featuring a custom automatic differentiation system, and advancing towards optimized attention computation in NumPy for efficient CPU execution.

A. Implementation Environment

The complete framework is implemented using the Python programming language [12]. The earlier implementations operate entirely without external machine learning libraries, while the optimized versions utilize NumPy [13] for efficient tensor computation and vectorized numerical operations.

The implementation environment consists of:

- Python 3.x,
- NumPy numerical library,
- CPU-only execution,
- lightweight file-based checkpoint storage,
- JSON and Pickle serialization.

The framework is tailored to operate on low resource systems, eliminating the need for GPU acceleration or distributed training infrastructure.

B. Project Structure

The μGPT framework is organized into multiple modular Python files, each responsible for a specific stage of transformer construction and inference.

The primary implementation modules include:

- `microgpt_base.py`
- `microgpt1.py`
- `microgpt2.py`
- `train.py`
- `chat.py`
- `model_runner.py`

The project directory additionally contains:

- dataset files,
- vocabulary mappings,
- trained model checkpoints,
- metadata configuration files.

C. Dataset Processing Pipeline

The framework accommodates datasets for both character level and word-level language modeling.

The early implementations use a names dataset containing more than 32,000 textual samples stored in `input.txt`. During preprocessing:

- duplicate whitespace is removed,

- documents are shuffled randomly,
- unique vocabulary symbols are extracted,
- token-index mappings are constructed.

The subsequent implementation based on NumPy also accommodates question-answer datasets by utilizing regex-based tokenization. The preprocessing pipeline:

- separates question-answer pairs,
- removes malformed samples,
- converts text into token sequences,
- generates fixed-length training windows.

The implementation of the tokenizer utilizes regular expressions to achieve token segmentation that is aware of punctuation.

`tokens = re.findall(r"\\w+[`^{ws}`]",`$text$`)|`

This method enables the framework to maintain punctuation marks while creating significant token boundaries.

D. Model Initialization

All trainable transformer parameters are initialized using randomized Gaussian or uniform distributions.

The initialized parameter matrices include:

- token embedding matrices,
- positional embedding matrices,
- query projection matrices,
- key projection matrices,
- value projection matrices,
- output projection matrices,
- feed-forward network weights.

The NumPy-based implementation initializes weights as:

$$W \sim U(-\alpha, \alpha) \tag{1}$$

where α represents the initialization scale factor.

The lightweight implementation uses relatively small embedding dimensions to maintain CPU efficiency:

- embedding dimension: 16–96,
- attention heads: 4,
- transformer layers: 1–2,
- sequence length: 12–16.

E. Custom Transformer Execution

The transformer forward pass is implemented manually without relying on external transformer libraries.

The forward pipeline consists of:

1) token embedding lookup,
2) positional embedding addition,
3) normalization,
4) self-attention computation,
5) residual connections,
6) feed-forward transformation,
7) output projection.

The self-attention implementation computes:

$$Q = XW_Q \tag{2}$$

$$K = XW_K \tag{3}$$

$$V = XW_V \tag{4}$$

followed by scaled attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{5}$$

The implementation clearly retains attention keys and values throughout autoregressive inference to maintain sequence context.

F. Autograd-Based Training

The early framework versions implement a fully custom scalar automatic differentiation engine through the `Value` class.

Each scalar value maintains:

- forward-computed numerical value,
- gradient value,
- references to parent computational nodes,
- local derivative information.

During training:

1) the forward computational graph is dynamically constructed,
2) loss values are computed using negative log likelihood,
3) reverse-mode automatic differentiation propagates gradients,
4) optimizer updates adjust model parameters.

The process of backward propagation navigates the computational graph in reverse topological order by applying the chain rule.

G. Optimization Strategy

The framework implements the Adam optimization algorithm [8] for parameter updates.

The optimizer maintains:

- first-moment moving averages,
- second-moment moving averages,
- bias correction terms.

Parameter updates are computed as:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{6}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{7}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{8}$$

Gradient clipping is additionally implemented to improve numerical stability:

$$g \leftarrow g \cdot \frac{\tau}{||g||} \tag{9}$$

where τ represents the clipping threshold.

Learning rate decay is incorporated during long training runs to stabilize convergence.

H. Dropout and Regularization

The stabilized framework versions implement dropout regularization [11] during training.

The dropout mechanism randomly suppresses activations with probability p:

$$x' = \begin{cases} 0, & \text{with probability } p \\ \frac{x}{1-p}, & \text{otherwise} \end{cases} \tag{10}$$

This improves generalization and reduces overfitting during transformer training.

I. Checkpointing and Model Serialization

The framework supports lightweight checkpoint storage using JSON and Pickle serialization formats.

The training pipeline periodically stores:

- model weights,
- vocabulary mappings,
- training metadata,
- optimizer configuration.

The NumPy-based implementation saves model files inside the `model/` directory:

- `weights.pkl`,
- `vocab.json`,
- `meta.json`.

Earlier implementations additionally support:

- `weights.json`,
- `checkpoint.json`.

These checkpoints enable continued training and standalone inference execution.

J. Inference Engine

The inference system is implemented through:

- `model_runner.py`,
- `chat.py`.

The inference pipeline performs:

1) prompt tokenization,
2) autoregressive forward propagation,
3) probability distribution generation,
4) probabilistic token sampling,
5) sequential response generation.

The conversational interface loads trained weights and generates responses iteratively using the trained transformer model.

K. Sampling Strategies

Multiple probabilistic decoding mechanisms are implemented across different framework versions:

- temperature sampling,
- top-k sampling,
- nucleus sampling,
- repetition penalty.

Top-k sampling restricts generation to the k highest-probability tokens:

$$P_k(x_i) = \frac{P(x_i)}{\sum_{j \in k} P(x_j)} \tag{11}$$

Nucleus sampling dynamically selects the smallest probability subset satisfying cumulative probability p.

Temperature scaling modifies output diversity:

$$P_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{12}$$

where T controls randomness during generation.

L. NumPy Acceleration

The final framework versions transition from scalar-only computation toward NumPy-accelerated tensor operations.

The vectorized implementation significantly improves:

- attention computation speed,
- matrix multiplication efficiency,
- training throughput,
- memory utilization.

The implementation based on NumPy maintains architectural transparency, allowing for practical experimentation with transformers on CPU systems.

M. Conversational Generation Module

The `chat.py` module implements an interactive conversational interface for the trained model.

The module:

- loads saved checkpoints,
- tokenizes user prompts,
- generates autoregressive responses,
- applies sampling controls,
- prevents repetitive token generation.

The conversational engine additionally incorporates:

- repetition penalties,
- configurable temperature scaling,
- top-k filtering,
- context window truncation.

These mechanisms improve generation quality and reduce unstable output behavior.

N. Implementation Summary

The suggested µGPT implementation illustrates that autoregressive language models based on transformers can be built from fundamental principles utilizing lightweight Python architectures. The framework effectively incorporates:

- custom autograd computation,
- transformer attention mechanisms,
- autoregressive language modeling,
- probabilistic text generation,
- lightweight checkpointing,
- CPU-based execution.

The final implementation functions as both an educational transformer framework and a research platform, allowing for the exploration of the internal workings of generative language models without relying on extensive deep learning infrastructures.

VI. Evaluation Setup

This section outlines the experimental setup, training environment, datasets, hyperparameter configurations, and evaluation methods employed to assess the performance and efficiency of the proposed µGPT framework.

A. Experimental Environment

All experiments were carried out using standard CPU execution without the aid of GPU acceleration. The implementation was fully developed in Python and NumPy to maintain the framework's lightweight and educational characteristics. The experiments took place on a Windows system utilizing Python 3.11.

In contrast to large-scale transformer systems that necessitate distributed GPU clusters and high-memory setups, µGPT was deliberately crafted for low-resource settings to illustrate that the principles of transformer-based language modeling can be effectively implemented and analyzed with minimal computational demands.

B. Datasets

Two different datasets were utilized during experimentation.

1) Character-Level Name Dataset: The first dataset consists of more than 32,000 human names stored in the `input.txt` file. This dataset is primarily used in the earlier versions of µGPT (`microgpt_base.py`, `microgpt1.py`, and `microgpt2.py`) for character-level autoregressive language modeling.

Each training sample is represented as a sequence of characters surrounded by a Beginning-of-Sequence (BOS) token:

$$x = [BOS, c_1, c_2, \ldots, c_n, BOS] \tag{13}$$

where c_i represents an individual character token.

The dataset enables the model to learn:

- character distributions,
- token transitions,
- sequential dependencies,
- and autoregressive generation behavior.

2) Healthcare Question–Answer Dataset: The advanced NumPy-based implementation (`train.py`) uses a healthcare question–answer dataset stored in:

`data/healthcare_qa_dataset.txt`

The dataset is automatically parsed into structured question–answer pairs using regular-expression-based preprocessing. Each training sample is transformed into the format:

$$\texttt{Question: } q \texttt{ Answer: } a \tag{14}$$

where q denotes the question text and a denotes the corresponding answer.

This dataset allows µGPT to perform token-level next-word prediction on semantically meaningful language sequences.

C. Tokenization Strategy

The framework employs lightweight rule-based tokenization.

For character-level training, every unique character becomes a vocabulary token.

For word-level training, regular-expression tokenization is applied:

```
re.findall(r"\w+|[^\w\s]", text.lower())
```

This tokenizer:

- preserves punctuation,
- separates symbols correctly,
- converts text to lowercase,
- and avoids dependency on external NLP libraries.

D. Training Configuration

The primary hyperparameters used in the NumPy-based µGPT training pipeline are summarized in Table I.

TABLE I: Training Hyperparameters

Parameter	Value
Sequence Length	12
Embedding Dimension	96
Learning Rate	0.003
Epochs	400
Learning Rate Decay	0.985
Minimum Learning Rate	5×10^{-4}
Gradient Clip Threshold	1.0
Attention Heads	4
Transformer Layers	2
Batch Size	4

The learning rate is decayed after each epoch according to:

$$\eta_t = \max(\eta_0 \times \gamma^t,\ \eta_{min}) \tag{15}$$

where:

- η_0 is the initial learning rate,
- γ is the decay factor,
- and η_{min} is the minimum learning rate threshold.

E. Optimization Method

The earlier versions of µGPT implement the Adam optimizer manually using pure Python scalar operations. The optimizer updates parameters using first-order and second-order moment estimates [8].

The parameter update rule is defined as:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \tag{16}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \tag{17}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{18}$$

where:

- g_t denotes the gradient,

- m_t and v_t represent moment estimates,
- and η is the learning rate.

Gradient clipping is additionally applied to stabilize training:

$$g \leftarrow g \cdot \frac{\tau}{\|g\| + \epsilon} \tag{19}$$

where τ is the clipping threshold.

F. Inference and Sampling

During the inference process, μGPT produces tokens in an autoregressive manner, utilizing the outputs generated earlier as context.

Different sampling strategies are used across versions:

- temperature sampling,
- top-k sampling,
- and nucleus (top-p) sampling.

The temperature-scaled probability distribution is computed as:

$$P(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{20}$$

where:

- z_i is the output logit,
- and T is the temperature parameter.

Lower temperatures produce more deterministic outputs, while higher temperatures increase generation diversity.

G. Evaluation Metrics

The evaluation of μGPT focuses primarily on qualitative and architectural analysis rather than large-scale benchmark accuracy.

The framework is evaluated using:

- training loss convergence,
- inference stability,
- generated text quality,
- architectural correctness,
- memory efficiency,
- and educational interpretability.

The cross-entropy loss function used during training is defined as:

$$\mathcal{L} = -\log P(y_t \mid x_{<t}) \tag{21}$$

where y_t denotes the target token and $x_{<t}$ denotes the previous context tokens.

H. Research Objective

The primary objective of μGPT is not to compete with billion-parameter industrial language models, but to provide:

- a transparent implementation of transformer internals,
- an educational research framework,
- a dependency-light architecture,
- and a fully interpretable GPT training pipeline.

The framework illustrates that contemporary transformer ideas can be constructed from fundamental principles utilizing concise and clear Python code, all while maintaining the core functionality of autoregressive language models.

VII. Experimental Results

This section outlines the experimental findings and qualitative outcomes derived from various iterations of the proposed μGPT framework. The experiments aimed to assess the learning ability, architectural integrity, training stability, inference quality, and computational efficiency of the lightweight transformer implementation.

A. Training Convergence

Throughout the training process, the model exhibited consistent convergence behavior in both character level and word level datasets. The training loss showed a steady decline as the number of optimization steps rose, suggesting that the transformer architecture effectively captured significant token dependencies.

The pure Python implementations (`microgpt_base.py`, `microgpt1.py`, and `microgpt2.py`) showed gradual convergence despite the absence of external deep learning frameworks.

The NumPy-based implementation (`train.py`) achieved significantly faster convergence due to vectorized tensor operations and optimized matrix computations.

Table II summarizes the observed convergence characteristics.

TABLE II: Training Behavior Across μGPT Variants

Model Version	Framework	Training Speed	Stability
microgpt_base	Pure Python	Low	Moderate
microgpt1	Pure Python	Moderate	Good
microgpt2	Pure Python	Moderate	Improved
train.py	NumPy	High	High

The introduction of:

- RMSNorm,
- dropout regularization,
- gradient clipping,
- improved parameter initialization,
- and learning-rate decay

significantly improved training stability in later versions of the framework.

B. Effect of Architectural Improvements

Multiple architectural refinements were introduced progressively throughout the development of μGPT.

1) Self-Attention Mechanism: The self-attention mechanism allowed the model to understand contextual relationships among tokens in a sequence. In contrast to conventional sequential architectures like recurrent neural networks, the transformer-based architecture showed enhanced contextual learning and the ability to perform parallel computations.

The attention computation is defined as:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{22}$$

Experimental findings indicated that the self-attention mechanism enhanced the coherence of sequences and the learning of token dependencies during inference.

2) Residual Connections: Residual skip connections improved gradient flow and stabilized deeper computations:

$$x_{out} = F(x) + x \tag{23}$$

Without residual connections, training became unstable and generated outputs degraded significantly.

3) RMS Normalization: RMSNorm enhanced the scaling of activations and minimized numerical instability throughout the optimization process. The normalization layer played a significant role in achieving smoother convergence and mitigating the issue of exploding gradients.

C. Inference Quality

After training, μGPT successfully generated realistic text patterns from both datasets.

1) Character-Level Generation: The character-level models created artificial names that closely mirrored actual names found in the dataset, all the while generating unique results.

Example generated outputs include:

TABLE III: Example Character-Level Outputs

Generated Samples
Aleron
Mikasha
Jenovia
Karveth
Samirya

The generated samples demonstrate that the model successfully learned:

- character distributions,
- phonetic structures,
- and common naming patterns.

2) Word-Level Generation: The implementation based on NumPy, which was trained using healthcare question answer data, generated responses that were semantically relevant to the given input context.

The generated outputs showed:

- contextual token prediction,
- punctuation-aware generation,
- and autoregressive continuation capability.

Despite being relatively smaller than industrial language models, the model effectively showcased fundamental GPT-style generative capabilities.

D. Sampling Strategy Analysis

Different sampling mechanisms produced noticeably different generation characteristics.

1) Temperature Sampling: Lower temperature settings produced consistent and predictable results, whereas higher temperature settings enhanced variability but also brought about instability.

The probability distribution during sampling is defined as:

$$P(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{24}$$

where:

- z_i denotes the logit,
- and T represents the temperature parameter.

Experimental observations indicated that temperature values between 0.6 and 0.8 produced the best balance between coherence and creativity.

2) Top-k Sampling: Top-k sampling restricted token selection to the highest-probability candidates, reducing noisy generations and improving output consistency.

3) Nucleus Sampling: The nucleus sampling implementation in `microgpt2.py` produced more natural outputs by dynamically selecting tokens from the cumulative probability distribution.

This approach improved:

- lexical diversity,
- contextual variability,
- and generation fluency.

E. Computational Efficiency

One of the primary goals of μGPT was computational minimalism.

The framework achieved:

- CPU-only execution,
- low memory consumption,
- dependency-free transformer training,
- and educational interpretability.

Unlike industrial transformer systems requiring GPUs and distributed training infrastructure, μGPT can be executed on standard consumer hardware.

Table IV summarizes the computational characteristics.

TABLE IV: Computational Characteristics of μGPT

Feature	μGPT
GPU Requirement	No
External DL Frameworks	No
CPU Execution	Yes
Custom Autograd Engine	Yes
Transformer Attention	Yes
Memory Footprint	Low
Educational Transparency	High

F. Comparison with Conventional Frameworks

Contemporary transformer frameworks like PyTorch and TensorFlow conceal the majority of their internal processes through high level APIs. Conversely, μGPT clearly delineates its implementations:

- scalar automatic differentiation,

- transformer attention,
- normalization,
- optimization,
- and autoregressive decoding

from first principles.

Although μGPT does not match the scale or performance of industrial language models such as GPT-2 [**?**] or GPT-3 [**?**], it provides significantly higher interpretability and educational value.

G. Research Observations

The experimental analysis demonstrates several important findings:

- Transformer architectures can be implemented successfully using lightweight pure Python code.
- Self-attention mechanisms substantially improve contextual sequence learning compared to simpler autoregressive methods.
- RMS normalization, residual connections, dropout, and gradient clipping collectively improve training stability.
- NumPy vectorization dramatically accelerates transformer computation while preserving architectural simplicity.
- Educational transformer implementations can effectively demonstrate modern language-modeling principles without requiring large-scale infrastructure.

H. Limitations

Despite its successful implementation, μGPT has several limitations:

- small parameter capacity,
- limited context window,
- absence of distributed training,
- no GPU acceleration,
- and limited benchmark-scale evaluation.

The framework prioritizes interpretability and educational clarity over large-scale performance optimization.

I. Summary

In summary, the experimental findings confirm the accuracy and efficacy of the proposed μGPT framework. This project effectively illustrates that contemporary transformer language models can be built from fundamental principles through compact, dependency-light implementations, all while preserving the key characteristics of autoregressive text generation and self-attention learning.

VIII. Discussion and Analysis

The experimental results demonstrate that μGPT successfully implements the core principles of transformer-based language modeling within a highly lightweight and interpretable framework. Despite being developed using pure Python and minimal NumPy operations, the system was capable of learning meaningful token relationships and generating coherent autoregressive outputs.

One of the most significant observations from this work is that modern transformer architectures can be reproduced without relying on large deep learning frameworks such as PyTorch or TensorFlow. The custom implementation of self-attention, RMS normalization, residual connections, dropout regularization, and the Adam optimizer validates the mathematical and architectural correctness of the framework.

The progression from `microgpt_base.py` to the optimized NumPy-based implementation also highlights the impact of architectural refinements on training stability and inference quality. The inclusion of gradient clipping, improved initialization, learning-rate decay, and advanced sampling strategies substantially improved convergence behavior and generation consistency.

Another important aspect of μGPT is its educational transparency. Unlike industrial-scale language models where most internal operations are abstracted behind framework-level APIs, μGPT exposes nearly every computational step explicitly. This makes the framework highly valuable for:

- academic study,
- transformer research,
- low-level neural network understanding,
- and experimentation with autoregressive architectures.

However, the framework also has several limitations. Due to its intentionally small parameter size and CPU-only execution, μGPT cannot compete with large-scale transformer systems in terms of accuracy, reasoning capability, or contextual understanding. The restricted context length and limited training data also constrain the complexity of generated outputs.

Nevertheless, the project successfully achieves its primary objective of demonstrating how transformer language models operate internally while maintaining implementation simplicity and computational accessibility. The framework establishes a strong foundation for future extensions involving larger datasets, multi-layer scaling, GPU acceleration, and more advanced transformer optimizations.

IX. Related Work

The creation of μGPT draws inspiration from numerous key research contributions in transformer architectures, neural language modeling, and deep learning frameworks for education.

The Transformer architecture proposed by Vaswani *et al.* [1] introduced the self-attention mechanism as an efficient alternative to recurrent computation for sequence modeling. This work established the foundation for modern Large Language Models (LLMs). Building upon this architecture, the GPT family of models demonstrated the effectiveness of autoregressive transformer-based language modeling for text generation and few-shot learning tasks [2]–[4].

Earlier neural language modeling approaches also significantly influenced the conceptual design of μGPT. Bengio *et al.* proposed one of the earliest neural probabilistic language models [14], while recurrent neural networks and Long Short-Term Memory (LSTM) architectures improved sequence learning capabilities before the widespread adoption of transformers [15], [16].

The educational and minimalist design philosophy of μGPT is strongly influenced by Andrej Karpathy's open-source educational projects, including *microgra*d, *makemore*, and *nanoGPT* [5]–[7]. These projects demonstrated that modern neural network systems can be implemented from first principles using compact and interpretable codebases while preserving conceptual clarity.

Several optimization and stabilization techniques integrated into μGPT are derived from established deep learning research. Layer Normalization [9], RMS Normalization [10], Dropout regularization [11], and the Adam optimization algorithm [8] collectively contribute to stable transformer training and improved convergence behavior.

In contrast to extensive industrial transformer frameworks that rely significantly on specialized hardware and deep learning libraries, μGPT emphasizes a lightweight implementation, transparency, and accessibility for educational purposes. This project integrates principles from contemporary GPT architectures with a design approach that minimizes dependencies, resulting in a research-focused transformer framework that is fully implemented in pure Python and NumPy.

X. COMPARATIVE STUDY

This section compares μGPT with several widely known educational and research-oriented language model frameworks. The comparison focuses on implementation complexity, dependency requirements, architectural transparency, and computational efficiency.

Table V presents the overall comparison.

TABLE V: Comparative Analysis of μGPT with Existing Educational GPT Frameworks

Framework	Language	External DL Framework	Custom Autograd	Transformer Attention	CPU Friendly	Educational Transparency
micrograd [5]	Python	No	Yes	No	Yes	Very High
makemore [7]	Python/PyTorch	Yes	No	Partial	Moderate	High
nanoGPT [6]	Python/PyTorch	Yes	No	Yes	Limited	Moderate
GPT-2 [3]	Python/TensorFlow	Yes	No	Yes	No	Low
μGPT (Proposed)	Python/NumPy	Minimal	Yes	Yes	Yes	Very High

Unlike industrial transformer implementations, μGPT prioritizes simplicity and interpretability over large-scale performance. While frameworks such as GPT-2 and nanoGPT are optimized for high-performance GPU training, they rely heavily on external deep learning ecosystems including PyTorch or TensorFlow.

In contrast, μGPT explicitly implements:

- automatic differentiation,
- self-attention,
- RMS normalization,
- residual learning,
- optimization,
- and autoregressive decoding

from first principles using lightweight Python code.

Compared to *micrograd*, μGPT extends the concept of custom autograd systems into transformer-based language modeling. Compared to *makemore*, μGPT introduces a more complete transformer architecture with multi-head self-attention and sequence-based contextual learning.

Although μGPT does not achieve the scale or performance of modern industrial LLMs, it provides significantly higher educational transparency and easier accessibility for students, researchers, and low-resource experimentation.

The comparative analysis demonstrates that μGPT occupies a unique position between educational neural-network implementations and practical transformer-based language modeling systems.

XI. CONCLUSION

This document introduces μGPT, a streamlined transformer-based language modeling framework created entirely with Python and NumPy, featuring minimal external dependencies. The initiative illustrates that the core concepts of contemporary GPT-style architectures can be constructed from foundational principles, all while maintaining transparency, interpretability, and computational efficiency.

The proposed framework successfully integrates key transformer components including:

- self-attention mechanisms,
- residual connections,
- RMS normalization,
- autoregressive token prediction,
- custom automatic differentiation,
- and Adam-based optimization.

Several iterations of μGPT were developed over time, starting with a completely dependency-free scalar autograd implementation and subsequently advancing to a more refined NumPy-based architecture that supports efficient CPU training and inference.

Experimental analysis revealed that μGPT is capable of understanding significant token relationships and producing coherent outputs for both character-level and word-level language modeling tasks. The framework also exhibited enhanced training stability by integrating dropout regularization, gradient clipping, learning-rate decay, and sophisticated sampling techniques.

In contrast to industrial large language models that emphasize scale and performance, μGPT prioritizes educational clarity, architectural transparency, and lightweight experimentation. This framework allows students and researchers to explore the internal workings of transformer systems without the need for extensive deep learning infrastructures.

Future work may include:

- scaling the model to larger datasets,
- extending context length,
- integrating GPU acceleration,
- implementing causal masking,
- adding multi-layer transformer blocks,
- and evaluating the framework on standardized NLP benchmarks.

In summary, μGPT creates a concise and comprehensible research framework aimed at elucidating transformer-based language models, while also offering a user-friendly basis for upcoming experiments in lightweight generative AI systems.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[2] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, Tech. Rep., 2018.

[3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019.

[4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[5] A. Karpathy, "micrograd," https://github.com/karpathy/micrograd, 2022.

[6] ——, "nanogpt," https://github.com/karpathy/nanoGPT, 2023.

[7] ——, "makemore," https://github.com/karpathy/makemore, 2022.

[8] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, 2015.

[9] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[10] B. Zhang and R. Sennrich, "Root mean square layer normalization," *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[11] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[12] Python Software Foundation, "Python programming language," https://www.python.org/, 2024.

[13] NumPy Developers, "Numpy," https://numpy.org/, 2024.

[14] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A neural probabilistic language model," *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003.

[15] T. Mikolov, M. Karafiat, L. Burget, J. Cernocky, and S. Khudanpur, "Recurrent neural network based language model," in *INTERSPEECH*, 2010.

[16] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[17] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[18] H. Zhang, Y. N. Dauphin, and T. Ma, "Fixup initialization: Residual learning without normalization," in *International Conference on Learning Representations (ICLR)*, 2019.